

The table of 1's and 0's could then be converted to **And** and **Or** blocks by the methods described above.

By adding two more inputs to the block, we could use similar techniques to build a block that not only adds but also subtracts, multiplies, and divides. The two extra control inputs would specify which of these operations was to take place. For instance, on every line of the table where the control inputs were 01, we would specify the output to be the sum of the input numbers, whereas in every combination where the control inputs were 10, we would specify the outputs to be the product, and so on. Most computers have logical blocks of this type inside them called *arithmetic units*.

Combining **And**s and **Or**s according to this strategy is one way to build any logical function, but it is not always the most efficient way. Often, by clever design, you can implement a circuit using far fewer building blocks than the preceding strategy requires. It may also be desirable to use other types of building blocks or to design circuits that minimize the delay from input to output. Here are some typical puzzles in logic design: How do you use **And** blocks and **Inverters** to construct **Or** blocks? (Easy.) How do you use a collection of **And** and **Or** blocks, plus only two **Inverters**, to construct the function of three **Inverters**? (Hard, but possible.) Puzzles like this come up in the course of designing a computer, which is part of what makes the process fun.

FINITE-STATE MACHINES

The methods I've described can be used to implement any function that stays constant in time, but a more interesting class of functions are those that involve sequences in time. To handle such functions, we use a device called a *finite-state machine*. Finite-state machines can be used to implement time-varying functions—functions that depend not just on the current input but also on the previous history of inputs. Once you learn to recognize a finite-state machine, you'll notice them everywhere—in combination locks, ballpoint pens, even legal contracts. The basic idea of a finite-state machine is to combine a look-up table, constructed using Boolean logic, with a memory device. The memory is used to store a summary of the past, which is the *state*

of the finite-state machine.

A combination lock is a simple example of a finite-state machine. The state of a combination lock is a summary of the sequence of numbers dialed into the lock. The lock doesn't remember all the numbers that have ever been dialed into it, but it does remember enough about the most recent numbers to know when they form the sequence that will open the lock. An even simpler example of a finite-state machine is the retractable ballpoint pen. This finite-state machine has two possible states—extended and retracted—and the pen remembers whether its button has been pressed an odd or an even number of times. All finite-state machines have a fixed set of possible states, a set of allowable inputs that change the state (clicking a pen's button, or dialing a number into a combination lock), and a set of possible outputs (retracting or extending the ballpoint, opening the lock). The outputs depend only on the state, which in turn depends only on the history of the sequence of inputs.

Another simple example of a finite-state machine is a counter, such as the tally counter on a turnstile indicating the number of people who have passed through. Each time a new person goes through, the counter's state is advanced by one. The counter is a *finite* state because it can only count up to a certain number of digits. When it reaches its maximum count—say, 999—the next advance will cause it to return to zero. Odometers on automobiles work like this. I once drove an old Checker cab with an odometer that read 70,000, but I never knew if the cab had traveled 70,000 miles, 170,000 miles, or 270,000 miles, because the odometer had only 100,000 states; all those histories were equivalent as far as the odometer was concerned. This is why mathematicians often define a state as “a set of equivalent histories.”

Other familiar examples of finite-state machines include traffic lights and elevator-button panels. In these machines, the sequence of states is controlled by some combination of an internal clock and input buttons such as the “Walk” button at the crosswalk and the elevator call and floor-selection buttons. The next state of the machine depends not only on the previous state but also on the signals that come from the input button. The transition from one state to another is determined by a fixed set of rules, which can be summarized by a simple state diagram showing the transition between states. [Figure 13](#) shows a state diagram for a traffic-light controller at an intersection where the light turns red in both directions after the Walk button is pressed. Each drawing of light represents a state and each arrow represents a transition

between states. The transition depends on whether or not the “walk” button is pressed.

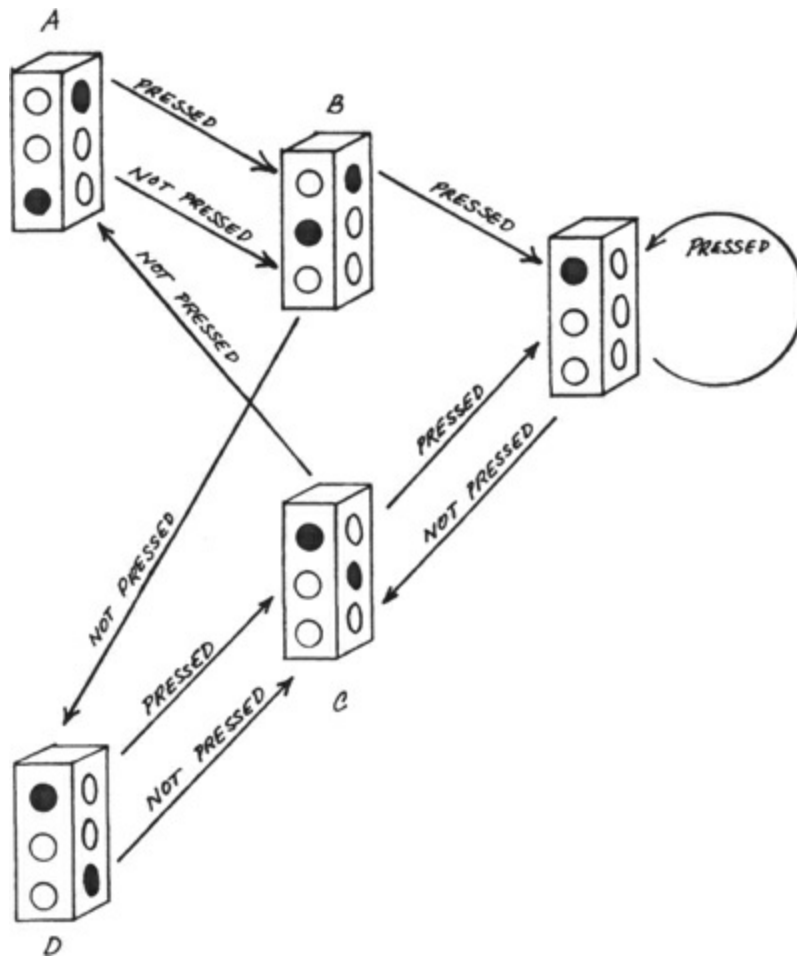


FIGURE 13

State diagram for a traffic-light controller

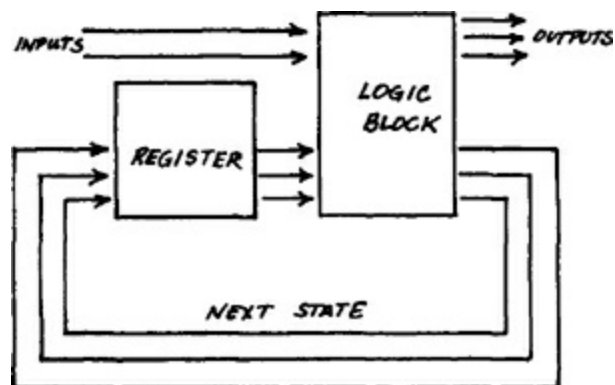


FIGURE 14

Finite-state machine, with logic block feeding register

To store the state of the finite-state machine, we need to introduce one last building block—a device called a *register*, which can be used to store bits. An n -bit register has n inputs and n outputs, plus an additional timing input that tells the register when to change state. Storing new information is called “writing” the state of the register. When the timing signal tells the register to write a new state, the register changes its state to match the inputs. The outputs of the register always indicate its current state. Registers can be implemented in many ways, one of which is to use a Boolean logic block to steer the state information around in a circle. This type of register is often used in electronic computers, which is why they lose track of what they’re doing if their power is interrupted.

A finite-state machine consists of a Boolean logic block connected to a register, as shown in [Figure 14](#). The finite-state machine advances its state by writing the output of the Boolean logic block into the register; the logic block then computes the next state, based on the input and the current state. This next state is then written into the register on the next cycle. The process repeats in every cycle.

The function of a finite-state machine can be specified by a table that shows, for every state and every input, the state that follows. For example, we can summarize the operation of the traffic-light controller by the following table:

<i>Inputs:</i>			<i>Outputs:</i>	
Walk Button	Current State	Main Road	Cross Road	Next State
Not Pressed	A	Red	Green	B
Not Pressed	B	Red	Yellow	D
Not Pressed	C	Yellow	Red	A
Not Pressed	D	Green	Red	C
Not Pressed	Walk	Walk	Walk	D
Pressed	A	Red	Green	B
Pressed	B	Red	Yellow	Walk
Pressed	C	Yellow	Red	Walk
Pressed	D	Green	Red	C
Pressed	Walk	Walk	Walk	Walk

The first step in implementing a finite-state machine is to generate such a table. The second step is to assign a different pattern of bits to each state. The five states of the traffic-light controller will require three bits. (Since each bit doubles the number of possible patterns, it is possible to store up to 2^n states using n bits.) By consistently replacing each word in the preceding table with a binary pattern, we can convert the table to a function that can be implemented with Boolean logic.

In the traffic-light system, a timer controls the writing of the register, which causes the state to change at regular intervals. Another example of a finite-state machine that advances its state at regular intervals is a digital clock. A digital clock with a seconds indicator can be in one of $24 \times 60 \times 60 = 86,400$ possible display states—one for each second of the day. The timing mechanism within the clock causes it to advance its state exactly once per second. Many other types of digital computing devices, including most general-purpose computers, also advance their state at regular intervals, and the rate at which they advance is called the *clock rate* of the machine. Within a computer, time is not a continuous flow but a fixed sequence of transitions between states. The clock rate of the computer determines the rate of these transitions, hence the correspondence between physical and computational time. For instance, the laptop computer on which I am writing this book has a clock rate of 33 megahertz, which means that it advances its state at a rate of 33 million times per second. The computer would be faster if the clock rate were higher, but its speed is limited by the time required for information to propagate through the logic blocks to compute the next state. As technology improves, the logic tends to become faster and the clock rate increases. As I write these words, my computer is state-of-the-art, but by the time you read this book computers with 33 megahertz clock rates will probably be considered slow. This is one of the wonders of silicon technology: as we learn to make computers smaller and smaller, the logic becomes faster and faster.

One reason finite-state machines are so useful is that they can recognize sequences. Consider a combination lock that opens only when it is given the sequence 0–5–2. Such a lock, whether it is mechanical or electronic, is a finite-state machine with the state diagram shown in [Figure 15](#).

A similar machine can be constructed to recognize any finite sequence.

Finite-state machines can also be made to recognize sequences that match certain patterns. Figure 16 shows one that recognizes any sequence starting with a 1, followed by a sequence of any number of 0s, followed by a 3. Such a combination will unlock the door with the combination 1–0–3, or a combination such as 1–0–0–0–3, but *not* with the combination 1–0–2–3, which doesn't fit the pattern. A more complex finite-state machine could recognize a more complicated pattern, such as a misspelled word within a stream of text.

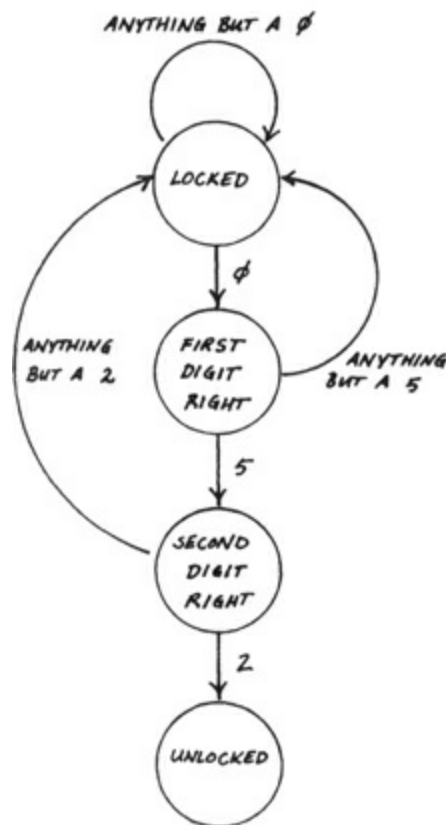


FIGURE 15

State diagram for a lock with combination 0-5-2

As powerful as they are, finite-state machines are not capable of recognizing all types of patterns in a sequence. For instance, it is impossible to build a finite-state machine that will unlock a lock whenever you enter any palindrome—a sequence that is the same forward and backward, like 3–2–1–1–2–3. This is because palindromes can be of any length, and to recognize

the second half of a palindrome you need to remember every character in the first half. Since there are infinitely many possible first halves, this would require a machine with an infinite number of states.

A similar argument demonstrates the impossibility of building a finite-state machine that recognizes whether a given English sentence is grammatically correct. Consider the simple sentence “Dogs bite.” The meaning of this sentence can be changed by putting a qualifier between the noun and the verb; for instance, “Dogs that people annoy bite.” This sentence can in turn be modified by putting another phrase in the middle: “Dogs that people with dogs annoy bite.” Although the meaning of such sentences might be expressed more clearly, and although they become increasingly difficult to understand, they are grammatically correct. In principle, this process of nesting phrases inside of one another can go on forever, producing absurd sentences like “Dogs that dogs that dogs that dogs annoy ate bit bite.” Recognizing such a sentence as grammatically correct is impossible for a finite-state machine, and for exactly the same reason it’s difficult for a person: you need a lot of memory to keep track of all those dogs. The fact that human beings seem to have trouble with the same kinds of sentences that stump finite-state machines has caused some people to speculate that we may have something like a finite-state machine inside our head for understanding language. As you will see in the next chapter, there are other types of computing devices that seem to fit even more naturally with the recursive structure of human grammar.

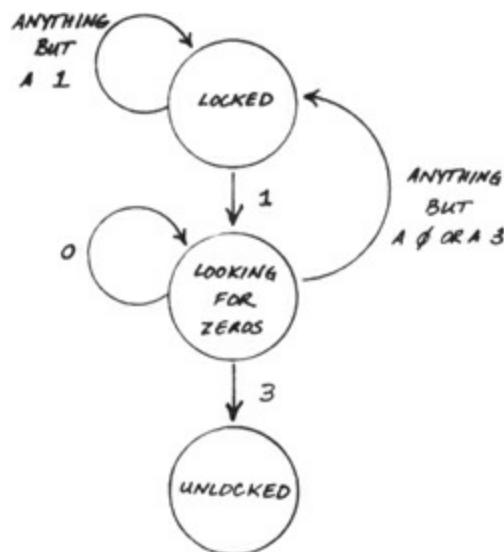


FIGURE 16

State diagram to recognize sequences like 1,0,3 and 1,0,0,0,:

I was introduced to finite-state machines by my mentor Marvin Minsky. He presented me with the following famous puzzle, called the *firing squad problem*: You are a general in charge of an extremely long line of soldiers in a firing squad. The line is too long for you to shout the order to “fire,” and so you must give your order to the first soldier in the line, and ask him to repeat to the next soldier and so on. The hard part is that all the soldiers in the line are supposed to fire at the same time. There is a constant drumbeat in the background; however, you can’t even specify that the men should all fire after a certain number of beats, because you don’t know how many soldiers are in the line. The problem is to get the entire line to fire simultaneously; you can solve it by issuing a complex set of orders which tells each soldier what to say to the soldiers on either side of him. In this problem, the soldiers are equivalent to a line of finite-state machines with each machine advancing its state by the same clock (the drumbeat), and each receiving input from the output of its immediate neighbors. The problem is therefore to design a line of identical finite-state machines that will produce the “fire” output at the same time in response to a command supplied at one end. (The finite-state machines at either end of the line are allowed to be different from the others.) I won’t spoil the puzzle by giving away the solution, but it can be solved using finite-state machines that have only a few states.

Before showing you how Boolean logic and finite-state machines are combined to produce a computer, I’ll skip ahead in this bottom-up description and tell you where we’re going. The next chapter starts by setting out one of the highest levels of abstraction in the function of a computer, which is also the level at which most programmers interact with the machine.